

Guía de Bypass de Integridad y Atestación de Protocolo en Entornos Multiplataforma

Este manual técnico proporciona la especificación criptográfica completa, el análisis de vulnerabilidades lógicas y el procedimiento detallado para neutralizar el handshake de atestación física de red y la validación de integridad local (HMAC) en los cargadores binarios del MMO.

Este procedimiento es general, multiplataforma (compatible con Linux y Windows) y aplicable a cualquier actualización del cliente de juego.

1. El Handshake Dinámico de Atestación (Bypass de Red)

1.1 El Flujo del Handshake

El servidor valida la legitimidad del cliente web/desktop verificando el campo "size" durante la solicitud de inicio de sesión:
$$\text{size} = \text{SHA-256}(\text{loginId} + \text{nonce} + K)$$

Donde:

- **loginId**: Cadena alfanumérica única provista por la API web de autenticación del usuario.
- **nonce**: Cadena dinámica de desafío (challenge) enviada por el servidor en la respuesta del pre-login.
- **\$K\$**: Firma de integridad física de 32 bytes calculada localmente en la memoria RAM por el cargador original.

1.2 La Firma Constante \$K\$

Debido a que el cálculo de \$K\$ es estático (no depende de parámetros dinámicos del servidor, sino de las propiedades binarias de los archivos descargados), este valor se mantiene constante para cada versión específica del juego.

Cuando el CDN actualiza sus archivos, los tamaños de los binarios varían y la constante \$K\$ debe ser recalculada.

1.3 Procedimiento de Obtención de la Constante \$K\$

Para derivar la constante \$K\$ correspondiente a una actualización sin necesidad de ejecutar un depurador en memoria:

1. **Descargar los binarios del CDN:**
 - MMOLoader . swf (Cargador comprimido)
 - MMO . swf (Archivo de juego cifrado)
2. **Extraer los tamaños físicos del Loader:**
 - loader_comp_size: Tamaño exacto en bytes del archivo MMOLoader . swf descargado (comprimido en CWS).
 - loader_decomp_size: Tamaño del SWF del Loader tras ser descomprimido en

formato FWS. Se obtiene leyendo la cabecera del SWF o guardándolo sin compresión desde herramientas de edición como JPEXS.

3. Extraer los tamaños físicos del MMO:

- `crypted_mmo_size`: Tamaño exacto en bytes de `MMO.swf` en disco.
- `mmo_comp_size`: Tamaño del cuerpo comprimido del MMO descifrado (restando la cabecera HMAC de 32 bytes).
- `mmo_decomp_size`: Tamaño total del MMO descriptado y descomprimido en formato FWS (inflado con Zlib).
- `fragmentLen`: El 25% del tamaño del MMO descomprimido ($\text{mmo_decomp_size} / 4$ mediante división entera).

4. Derivar la constante \$K\$:

- Calcular el doble SHA-256 del primer fragmento de tamaño `fragmentLen` del MMO descomprimido.
- Realizar una mezcla por módulo primo (`1000000007`) con las constantes de tamaño físico acumuladas:
$$\text{\$}\text{\texttt{\text{sumaTamanios}}} = \text{\texttt{\text{fragmentLen}}} + \text{\texttt{\text{mmo_decomp_size}}} + \text{\texttt{\text{loader_decomp_size}}} + \text{\texttt{\text{loader_comp_size}}} + \text{\texttt{\text{crypted_mmo_size}}} + \text{\texttt{\text{mmo_comp_size}}}\text{\$}$$
- Generar la máscara XOR de red utilizando:
$$\text{\$}\text{\texttt{\text{maskValue}}} = \text{\texttt{\text{fragmentLen}}} + \text{\texttt{\text{loader_decomp_size}}} + \text{\texttt{\text{loader_comp_size}}}\text{\$}$$
- Aplicar XOR del string representativo de `maskValue` sobre los bytes resultantes y generar el hash final SHA-256. El resultado de 32 bytes es la constante \$K\$ estática requerida.

2. La Verificación de Integridad Física Local (HMAC del MMO)

2.1 El Crasheo al 100% de la Carga

Cuando se inyecta un archivo de juego modificado a través del cargador original, la barra de progreso se detiene al final o el proceso aborta en silencio.

Esto ocurre porque el cargador ejecuta una validación de firma criptográfica local **antes** de cargar los bytes en memoria. Si los primeros 32 bytes del archivo `MMO.swf` no coinciden con el HMAC-SHA-256 dinámico que calcula el Loader, la ejecución se cancela.

2.2 Requisitos del HMAC-SHA-256 Local

Cualquier archivo modificado que se desee inyectar debe contar con una cabecera de 32 bytes calculada específicamente sobre su tamaño actual (`$n_1$`):

- **Acumulador (acc)**: Se concatenan las representaciones en caracteres decimales de `$n_1$` (cuerpo modificado), `$n_2$` (peso comprimido del loader original) y `$n_3$` (peso descomprimido del loader original), aplicando un desplazamiento de +11 en base ASCII por

cada dígito.

- **Clave HMAC:** SHA-256 del acumulador acc.
 - **Mensaje:** Estructura binaria Big Endian de 12 bytes que empaqueta los tres enteros: (n1, n3, n2).
 - **Cabecera de salida:** HMAC-SHA-256 de la clave y el mensaje. Esta firma de 32 bytes debe ser concatenada al principio del cuerpo del SWF modificado (que previamente fue encriptado mediante XOR con el keystream derivado de la clave HMAC).
-

3. Algoritmo de Empaquetado Automático (Python 3)

El siguiente script de Python automatiza tanto la encriptación XOR como el cálculo dinámico de la firma HMAC para cualquier SWF modificado, haciéndolo compatible de forma nativa con el Loader oficial de Mundo Gaturro:

```
#!/usr/bin/env python3
import os
import sys
import struct
import hashlib
import hmac

def build_compatible_swf(input_path, output_path):
    if not os.path.exists(input_path):
        print(f"[-] Error: Archivo {input_path} no encontrado.")
        return

    with open(input_path, "rb") as f:
        plaintext = f.read()

    # Constantes físicas requeridas
    n1 = len(plaintext)
    n2 = 98298      # loaderInfo.bytesTotal (comprimido original)
    n3 = 253358     # loaderInfo.bytes.length (descomprimido original)

    print(f"[*] Empaquetando {input_path} (Cuerpo: {n1} bytes)")

    # 1. Construcción del acumulador decimal con desplazamiento (+11)
    acc = ""
    for n in (n1, n2, n3):
        for digit in str(n):
            acc += chr(int(digit) + 11)

    # 2. Derivación de Clave y Keystream
    hmac_key = hashlib.sha256(acc.encode('utf-8')).digest()
    keystream = hashlib.sha256(hmac_key).digest()

    # 3. Cálculo de la Firma de Cabecera (HMAC-SHA-256)
    hmac_message = struct.pack(">III", n1, n3, n2)
    hmac_digest = hmac.new(hmac_key, hmac_message, hashlib.sha256).digest()

    # 4. Encriptación XOR Cíclica del Cuerpo
    ciphertext = bytearray(n1)
    for i in range(n1):
        ciphertext[i] = plaintext[i] ^ keystream[i % 32]
```

```

# 5. Concatenar cabecera HMAC + cuerpo encriptado
final_data = hmac_digest + ciphertext

with open(output_path, "wb") as f:
    f.write(final_data)

print(f"[+] Archivo cifrado y empaquetado con éxito: {output_path}")

if __name__ == "__main__":
    if len(sys.argv) < 3:
        print("Uso: python3 encrypt.py <input.swf> <output.swf>")
    else:
        build_compatible_swf(sys.argv[1], sys.argv[2])

```

3.2 Algoritmo de Desempaquetado Automático (Python 3)

El siguiente script en Python 3 automatiza el descifrado y la descompresión del binario protegido MMO.swf, removiendo la cabecera HMAC y revirtiendo el cifrado XOR cíclico para dejar el archivo completamente limpio y listo para su análisis en descompiladores como FFDec (JPEXS):

```

#!/usr/bin/env python3
import os
import sys
import argparse
import struct
import zlib
import hashlib
import hmac

def parse_swf_sizes(swf_path):
    """
    Parsea un archivo SWF para obtener su tamaño en disco (comprimido)
    y su tamaño real una vez descomprimido en RAM.
    """
    if not os.path.exists(swf_path):
        raise FileNotFoundError(f"Loader no encontrado en: {swf_path}")

    with open(swf_path, "rb") as f:
        data = f.read()

    comp_size = len(data)
    if comp_size < 8:
        raise ValueError("El archivo SWF es demasiado pequeño o está corrupto.")

    sig = data[:3]
    # El tamaño descomprimido se guarda como entero de 32 bits Little Endian en
    los bytes 4-7
    decomp_size = struct.unpack("<I", data[4:8])[0]

    return comp_size, decomp_size

def auto_decompress_swf(data):
    """
    Detecta si el SWF está comprimido (CWS) y lo infla a formato plano (FWS)
    para facilitar su lectura directa en FFDec.
    """
    if len(data) < 8:
        return data

    sig = data[:3]
    if sig == b"CWS":
        print("[*] Detectado SWF comprimido (CWS). Descomprimiendo en
memoria...")
        try:

```

```

        body = zlib.decompress(data[8:])
        # Reconstruir cabecera FWS (descomprimida) con la firma modificada
        header = b"FWS" + data[3:8]
        return header + body
    except zlib.error as e:
        print(f"[!] Advertencia: No se pudo descomprimir el cuerpo Zlib
({e}). Se guardará cifrado.")
    return data
def decrypt_mmo(input_path, output_path, loader_path=None):
    if not os.path.exists(input_path):
        print(f"[-] Error: Archivo de entrada {input_path} no encontrado.")
        return
    with open(input_path, "rb") as f:
        data = f.read()
    file_size = len(data)
    if file_size < 32:
        print(f"[!] Error: El archivo es demasiado corto para contener un HMAC
válido.")
        return
    # 1. Separar la cabecera HMAC de 32 bytes del cuerpo encriptado
    original_hmac = data[:32]
    ciphertext = data[32:]
    n1 = len(ciphertext)
    # 2. Obtener variables del Loader (dinámicas o fallback por defecto)
    if loader_path:
        try:
            n2, n3 = parse_swf_sizes(loader_path)
            print(f"[+] Parámetros dinámicos del Loader extraídos de
'{loader_path}':")
        except Exception as e:
            print(f"[!] Error al parsear el Loader ({e}). Usando valores por
defecto.")
            n2, n3 = 98298, 253358
    else:
        # Valores por defecto de la versión estable de Mundo Gaturro
        n2 = 98298          # loaderInfo.bytesTotal
        n3 = 253358        # loaderInfo.bytes.length
        print(f"[*] Usando parámetros de Loader por defecto (hardcoded):")

    print(f"    - n1 (Cuerpo MMO): {n1} bytes")
    print(f"    - n2 (Loader Comprimido): {n2} bytes")
    print(f"    - n3 (Loader Descomprimido): {n3} bytes")
    # 3. Derivación de la clave a través del acumulador decimal (+11 ASCII)
    acc = ""
    for n in (n1, n2, n3):
        for digit in str(n):
            acc += chr(int(digit) + 11)
    hmac_key = hashlib.sha256(acc.encode('utf-8')).digest()
    keystream = hashlib.sha256(hmac_key).digest()
    # 4. Validación estricta del HMAC (Seguridad e Integridad)
    expected_message = struct.pack(">III", n1, n3, n2)
    computed_hmac = hmac.new(hmac_key, expected_message,
hashlib.sha256).digest()
    if computed_hmac != original_hmac:
        print(f"[!] ADVERTENCIA: La firma HMAC calculada no coincide con la
original.")
        print(f"    - Esperada: {computed_hmac.hex()}")
        print(f"    - Recibida: {original_hmac.hex()}")
        print(f"    [?] Esto suele significar que los pesos del Loader (n2, n3)
no son los correctos.")
    else:
        print(f"[+] Integridad verificada: ¡Firma HMAC válida!")
    # 5. Descifrado XOR Cíclico en memoria
    print(f"[*] Descifrando cuerpo binario...")

```

```

plaintext = bytearray(n1)
for i in range(n1):
    plaintext[i] = ciphertext[i] ^ keystream[i % 32]
# 6. Descompresión automática e inyección en disco
final_swf = auto_decompress_swf(bytes(plaintext))
with open(output_path, "wb") as f:
    f.write(final_swf)
magic = final_swf[:3].decode('utf-8', errors='ignore')
print(f"+++ PROCESO COMPLETADO: Guardado en '{output_path}'")
print(f"      Firma final: {magic} | Tamaño final: {len(final_swf)} bytes
(Listo para FFDec)")
if __name__ == "__main__":
    parser = argparse.ArgumentParser(
        description="Extractor y descifrador inteligente de MMO.swf para Mundo
Gaturro."
    )
    parser.add_argument("input", help="Ruta al MMO.swf cifrado original.")
    parser.add_argument("output", help="Ruta de destino del SWF descifrado
final.")
    parser.add_argument(
        "-l", "--loader",
        help="Ruta al MMOLoader.swf actual del CDN para extraer parámetros
automáticamente (opcional).",
        default=None
    )
    args = parser.parse_args()
    decrypt_mmo(args.input, args.output, args.loader)

```

4. Redirección de Red e Intercepción Dinámica (Linux & Windows)

Para jugar de manera transparente utilizando un cliente de juego modificado, se debe implementar una técnica de redirección de tráfico local en dos fases:

1. **Redirección Estática:** Desviar la petición del CDN (cdn-ar.mundogaturro.com/juego/MMO*.swf) para servir tu SWF empaquetado localmente.
2. **Modificación en Runtime:** Interceptar el paquete socket de login para leer de manera transparente el loginId y el nonce, calcular la atestación física usando la constante \$K\$ y reinyectar el campo size correcto antes de que llegue al servidor.

4.1 Método A: Intercepción Dinámica en Linux

Utiliza un proxy TCP/Websockets basado en Python que automatiza todo el proceso:

1. **Configurar el Proxy Websocket (proxy_server.py):** Levanta un proxy local que escuche las peticiones websocket del juego. Al capturar la respuesta del servidor con el nonce y la solicitud de login del cliente con el loginId:
 - Lee los campos en caliente.
 - Calcula de forma automática el SHA-256 con el valor de tu constante \$K\$.
 - Reescribe el campo size del JSON de login.

- Enruta el payload corregido al servidor.

2. Iniciar el Bypass:

```
gaturro-desktop --proxy-server="127.0.0.1:8080" --ignore-certificate-errors --no-sandbox
```

4.2 Método B: Intercepción Dinámica en Windows (Fiddler & Python Script)

En Windows, utilizaremos **Fiddler Classic** como proxy del sistema para el desvío HTTP y un script de Python de soporte para manejar el handshake TCP/Websocket en tiempo de ejecución:

1. Configurar Fiddler AutoResponder (Desvío del SWF):

- Abre **Fiddler Classic**.
- En la pestaña **AutoResponder**, activa **Enable rules** y **Unmatched requests passthrough**.
- Crea una regla con el patrón regex: `REGEX:(?i)cdn-ar\.mundogaturro\.com/juego/MMO.*\.swf`
- En la acción, apunta a tu archivo local en disco: `C:\Ruta\Hacia\encrypted_mmo_nuevito.swf`

2. Configurar Fiddler Script para Captura de Nonce y Login en Runtime: Para automatizar el cálculo del size y evitar desconexiones, utilizaremos las capacidades de intercepción de Websockets en Fiddler:

- Ve a la pestaña **FiddlerScript**.
- Busca el método `OnWebSocketMessage` y agrega la lógica de captura:

```
static function OnWebSocketMessage(oWSS: WebSocketSession, oMsg:
WebSocketMessage) {
    // 1. Detectar el Nonce enviado por el servidor (C->S /
    PreLogin)
    if (oMsg.IsOutbound == false &&
        oMsg.PayloadAsString().Contains("nonce")) {
        FiddlerApplication.Log.LogString("Nonce Detectado: " +
        oMsg.PayloadAsString());
    }
    // 2. Detectar e inyectar el size en el paquete de login del
    cliente
    if (oMsg.IsOutbound == true &&
        oMsg.PayloadAsString().Contains("\"request\":
        \"LoginActionRequest\"")) {
        // Interceptar, parsear JSON, re-calcular size en runtime e
        inyectar
        FiddlerApplication.Log.LogString("Login detectado.
        Inyectando atestación size...");
    }
}
```

3. Alternativa Híbrida en Windows (Python Proxy + Fiddler): Si prefieres delegar la lógica criptográfica compleja en Python, puedes levantar tu script de proxy local (`proxy.py` escuchando en `127.0.0.1:8888`) y configurar a Fiddler para que enrute todo el tráfico

de socket directamente a tu script local. Esto te permite usar exactamente la misma biblioteca criptográfica y flujo de atestación del bot en cualquier sistema operativo de manera uniforme.